Small Image Library Pure Next Submission

Live demo: https://small-image-library-pure-next-emhc.vercel.app/
GitHub repository: https://github.com/EugeneTolstikhin/SmallImageLibraryPureNext

Architecture Overview

Single Dockerized Next.js TypeScript application. The frontend lives in app/page.tsx,
and the backend search layer uses App Router route handlers under app/api/*.
This version follows the challenge wording closely: the search layer is inside Next.js.
There are no separate NestJS, Postgres, or Redis services.

Search Strategy

The app uses the two provided media examples plus generated fixtures to create 10,000 items.
Each item is normalized and loaded into an in-memory inverted index at runtime.
suchtext is weighted 5, fotografen is weighted 2, and bildnummer is weighted 8.
The index supports token and prefix matching, with date and relevance sorting.

Preprocessing

German dates are parsed from DD.MM.YYYY to ISO dates. Invalid dates become null, remain keyword-searchable, and are excluded from active date-range filters. Text is accent-normalized, lowercased, tokenized, and cleaned of noisy delimiters. Restrictions such as PUBLICATIONxINxGERxSUIxAUTxONLY are extracted as structured filter labels.

API And UI

The app exposes GET /api/search, GET /api/facets, and GET /api/analytics.
The search response includes items, page, pageSize, total, totalPages, and timingMs.
The Tailwind UI includes debounced search, credit filter, date range controls, restriction chips, sorting, result cards, snippets, loading, empty, error, and pagination states.

Analytics

The demo tracks total searches, recent query timings, and common normalized keywords in memory.

Scaling And Continuous Ingestion

For 10,000 records, an in-memory index is simple, transparent, and fast enough for the challenge. For millions of records, I would keep the API contract stable but move search indexing and ranking to Meilisearch, OpenSearch, or Elasticsearch. New items would be preprocessed, stored durably, indexed asynchronously, and trigger cache invalidation or index versioning.

Testing Approach

Automated tests cover date parsing, invalid dates, restriction extraction, accent normalization, tokenization, search, filters, sorting, pagination, parameter clamping, and XSS-safe snippets.

Trade-Offs

This version is directly aligned with the prompt and easy to run, but less production-like.
Data and analytics are in memory. A production version needs durable storage, async ingestion, a dedicated search backend, observability, integration tests, CI/CD, and hosted deployment.